Holly Pasquinelli
Lab 3

Disk Forensics

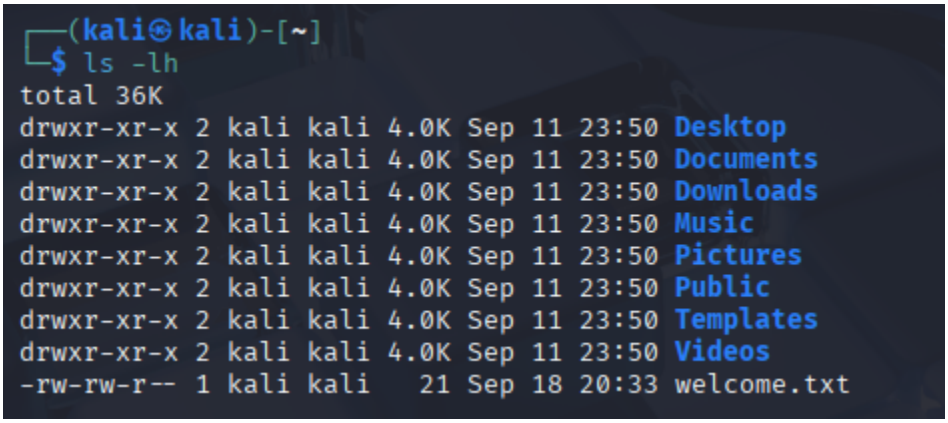
Non-technical Summary

In this lab we explored Linux commands that help to conduct analysis on drives within a system. Linux commands are a very powerful tool when it comes to digital forensics. We are able to investigate parts of file systems such as permissions, contents, filetypes, and hidden files. As we explored hashing in the last lab, in this lab we are also able to use hashing in the Linux command line itself to hash files, or even entire directories. Using the Linux command line we were also able to conduct analysis on drives and partitions. We explored commands that are helpful when working as an attacker or security professional. We also learned how to use Linux swap, where part of a hard disk is considered as an extension of RAM, or virtual RAM. These techniques and commands are helpful in digital forensics to analyze forensic images of disks, and understand how disk storage is partitioned. At the end of the lab, we also explored some of the PowerShell cmdlets for Windows disk investigations. These cmdlets can give investigators the power to understand boot code, uncover hidden data, and even signatures.

Technical Report

We started the lab by exploring chmod in class. The chmod command helps us to change file permissions of a file, folder, or pointer to another file. We use ls to list all directories, except for hidden ones. If we would like to view all files, including hidden directories and files, we can use ls -a. The hidden files will start with a ".".

We can use the -lh specification to view as human readable. As we can see below, the total size is in Kilobytes (K). We see in the first column there is a series of drwx. The first letter signifies the file type, "d" is for directory, or folder, "-" specifies a file, and "l" specifies a pointer, or link to another folder.



The next sets, combinations of rwx, specify file permissions. Read is “r”, write is “w”, execute is “x”. The first group, or first three characters after the filetype specifier represents the permissions for the user (u). The next three represent the permissions for the group to which the user belongs (g), and the third set represents the permissions for everyone else, or “others” (o). We can change these permissions using the chmod command, as previously mentioned.

For example, ch mod 666 changes the rw permissions for all groups, in binary 6 is 110. So we see 110, rwx, respectively. We can also change the permissions for all groups by using chmod -x. Using chmod u+x adds execute permission for the current user. Lastly, chmod o-w removes the writing permission for others.

In this lab we explored the sudo fdisk command. Using sudo fdisk -l, we can look for devices. We learned about Linux swap, where once a system runs out of RAM, we can take part of the hard disk, and consider it as an extension of RAM, virtual RAM. We see here when we run sudo fdisk that for this virtual machine, we have sda1 as the first partition. Where sda is the hard disk.

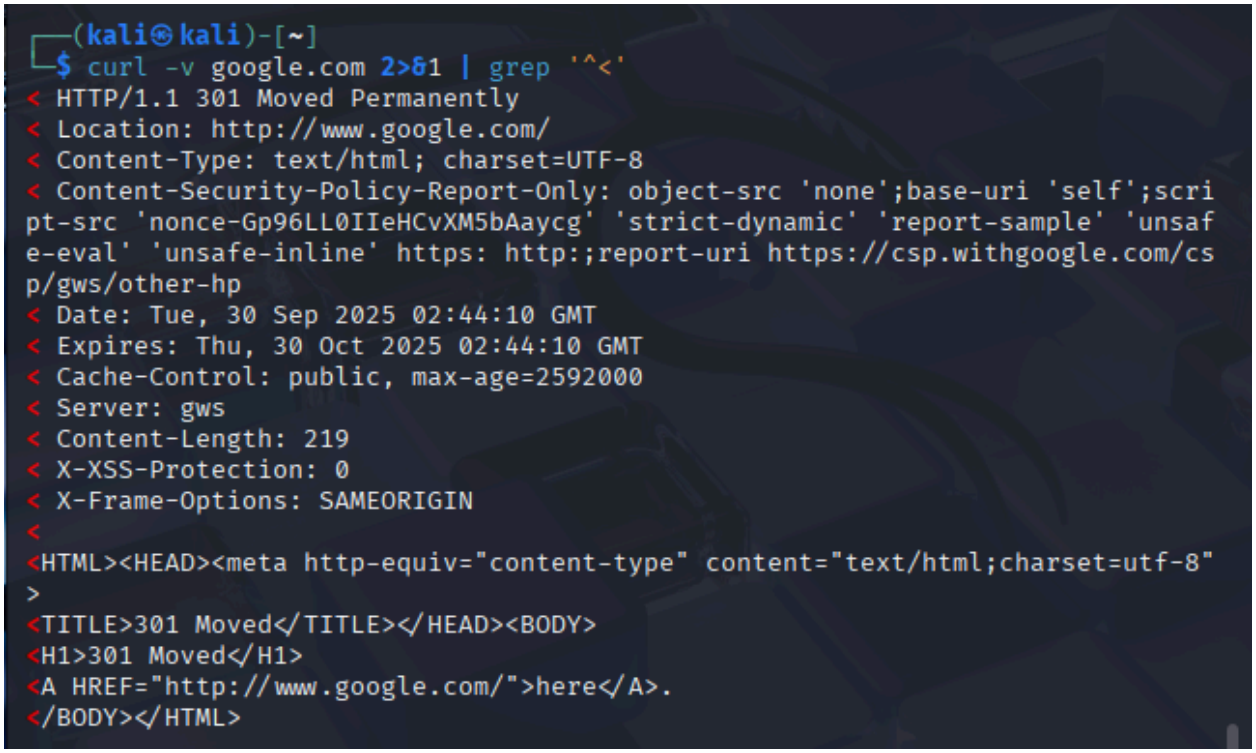
```
┌──(kali㉿kali)-[~]
└─$ sudo fdisk -l
[sudo] password for kali:
Disk /dev/sda: 80.09 GiB, 86000000000 bytes, 167968750 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x3a69c69b

Device     Boot Start       End   Sectors  Size Id Type
/dev/sda1  *     2048 167968749 167966702 80.1G 83 Linux
```

In the lab, we also discussed slack space in sectors. This means that in addition to space for the system and its contents, there is error correction code (ECC). This includes data redundancy to recover bits in the event that data is lost, and needs to be recovered. We can use the lshw volume command to show partitions, and get more information about them, such as a description of the device, and its class. This is helpful when there are multiple devices, and partitions.

Another important Linux command is curl. We can use curl to get website information. As we can see below, we use google.com, and pass it as stdin, using “2>&1”. We can use grep ‘^<’ to display only the lines starting with <. This allows us to look for more specific information. We can see below that it can show location, headers, status. In this command, we also use pipes “|”, which takes output and feeds it into the next command. This creates an efficient way to link multiple commands and receive and

search for additional information or look for matching queries within files, directories, or websites.



We learned that hashing on the Linux command line is very simple. We can simply use *sha1sum [filename]* to hash a file.

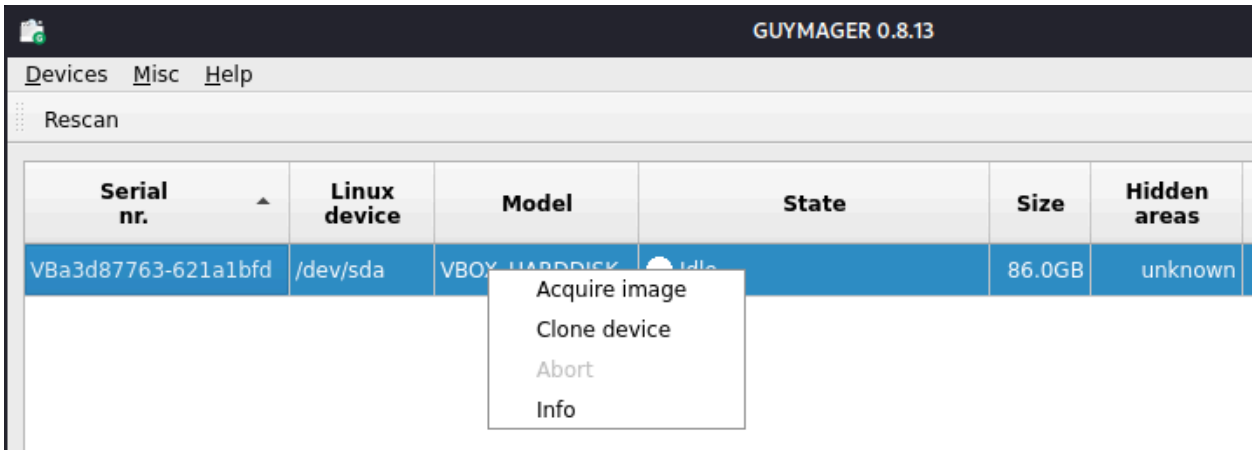
Another command we learned about is dc3dd, and dd commands. If there is a crime committed using a computer, we mount it, connect it to the hard disk, and copy it to the computer. Using the following command:

sudo dc3dd if=/dev/sdc hash=sha1 log=usb_forensics.log of=usb_image.dd

We can see the input file (if), the hard disk (dev/dsc), the hash where we ensure the evidence won't be modified later, the log which keeps as hash, and the file it is copied to (of). We can combine files using *cat* command. Using *cat* we can even hash files together so that all content is combined, and then the combined content is sent to be hashed. This can be achieved by using a pipe (|). dc3dd can also be used to completely wipe drive contents. This would avoid deleted files being recovered.

One last important tool we used in the Linux portion of this lab was Guymager. This is how we can create copies of disks, and create forensic disk images. Guymager creates copies of disks to avoid modifying original data, and ensure evidence is preserved, similar to hashing. In the image below, we can see that the device selected is the hard disk, sda. Once we select acquire image from the dropdown, we can create an exact copy to the bit. This saves evidence as an image file. We can use the dd

command to perform additional analysis, and retrieve the master boot record. This is stored in the first sector of the booting drive, again /dev/sda. The master boot record gives us block size, and count.



We can open the image generated by Guymager in HxD, which was used in the previous lab. HxD allows us to analyze the data in the file pixel by pixel. It is here where we can also look for signatures in the data as well.

Lastly, we looked into a few commands using Windows PowerShell. We used the PowerForensics module. Specifically, we used Get-ForensicVolumeBootRecord to return a master boot record for a volume. We can specify paths to volumes using \\\\., and we can change the representation of the output. In the lab we used -AsBytes for the raw version of the data, and then specified hexadecimal format. This makes analysis of the boot record easier to look at.

Get-ForensicFileRecord was used to get the master file table (MFT). We performed this command on an image file. The MFT gives us all information about every file and directory on the specified volume. In digital forensics, the MFT can give investigators information regarding changes in file systems, finding hidden files, and unusual activity on a system.